

State Estimation using Kalman Filter (State Estimation, KF, EKF, Damped Harmonic Oscillator, Cart-Pole)

Robotic Systems I

Konstantinos Asimakopoulos *k_asimakopoulos@ac.upatras.gr*
Lampros Printzios *printzios_lampros@ac.upatras.gr*

Department of Electrical and Computer Engineering
University of Patras

March 23, 2026

Table of Contents

1 State Estimation in Robotics

- What is it and How it Works in Robotics?

2 Discretization & Linearization

- Linear Discrete Systems
- Nonlinear Discrete Systems
- Linearization of Nonlinear Discrete Systems

3 Kalman Filter

- What is it?
- How it Works in Linear Systems (KF)?
- How it Works in Nonlinear Systems (EKF)?

4 Damped Harmonic Oscillator Exercise

- System Dynamics
- Discretized System
- Example

5 Cart-Pole Exercise

- System Dynamics
- Discretized System & Jacobians
- Example

6 References

State Estimation in Robotics - What is it and How it Works in Robotics?

In control, automation and robotics, **state estimation** aims to recover the **true internal state** $\mathbf{x}$ of a system from its known control inputs $\mathbf{u}$ and the noisy measurements $\mathbf{z}$ of its true outputs $\mathbf{y}$, while accounting for **motion/process noise** $\mathbf{w}$ and **observation/measurement noise** $\mathbf{v}$. The noise may come from many different sources, like *imperfect incomplete models*, *noisy sensors*, *unknown external disturbances*, etc, so we need a way to get *reliable state estimates* for control and decision making. The key idea is to combine:

- 1 a **motion/process model** (how the system moves forward / evolves)
- 2 an **observation/measurement model** (what the sensors observe)

General discrete-time state space model with additive Gaussian noise:

$$\begin{aligned}\mathbf{x}_k &= \mathbf{f}_d(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) + \mathbf{w}_{k-1} \\ \mathbf{z}_k &= \mathbf{h}_d(\mathbf{x}_k) + \mathbf{v}_k, \quad \mathbf{y}_k = \mathbf{h}_d(\mathbf{x}_k)\end{aligned}$$

Common estimation loop (used in Kalman Filter):

- 1 **Prediction:** use process model to predict next state
- 2 **Correction:** update estimation using sensor measurements

Discretization & Linearization - Linear Discrete Systems

Let's assume the classical **continuous-time state space model**:

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t)$$

$$\mathbf{y}(t) = \mathbf{C}\mathbf{x}(t) + \mathbf{D}\mathbf{u}(t)$$

We can analytically **discretize this linear system** with **sampling time dt** , assuming **zero-order hold** for the input (meaning it remains constant between successive time steps), as follows:

$$\mathbf{x}_{k+1} = \mathbf{A}_d\mathbf{x}_k + \mathbf{B}_d\mathbf{u}_k$$

$$\mathbf{y}_k = \mathbf{C}_d\mathbf{x}_k + \mathbf{D}_d\mathbf{u}_k$$

where:

$$\mathbf{A}_d = e^{\mathbf{A}dt}, \quad \mathbf{B}_d = \left(\int_0^{dt} e^{\mathbf{A}\tau} d\tau \right) \mathbf{B} = \mathbf{A}^{-1}(e^{\mathbf{A}dt} - \mathcal{I})\mathbf{B}, \text{ if } \exists \mathbf{A}^{-1}$$

$$\mathbf{C}_d = \mathbf{C}, \quad \mathbf{D}_d = \mathbf{D}$$

We could also compute discretizations using higher-order holds, like *first-order / piecewise-linear hold*.

Discretization & Linearization - Nonlinear Discrete Systems

For a **nonlinear continuous-time system**:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t))$$

In general, we **cannot compute an analytical discretization**, so we use **numerical integration methods** to approximate:

$$\mathbf{x}_{k+1} \approx \mathbf{f}_d(\mathbf{x}_k, \mathbf{u}_k)$$

Explicit Euler (1st order), very simple but not so accurate:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + dt \cdot \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)$$

Runge-Kutta 4th order (RK4), more accurate and widely used in practice:

$$\mathbf{k}_1 = \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k)$$

$$\mathbf{k}_2 = \mathbf{f}\left(\mathbf{x}_k + \frac{dt}{2} \cdot \mathbf{k}_1, \mathbf{u}_k\right)$$

$$\mathbf{k}_3 = \mathbf{f}\left(\mathbf{x}_k + \frac{dt}{2} \cdot \mathbf{k}_2, \mathbf{u}_k\right)$$

$$\mathbf{k}_4 = \mathbf{f}(\mathbf{x}_k + dt \cdot \mathbf{k}_3, \mathbf{u}_k)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{dt}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4)$$

Discretization & Linearization - Linearization of Nonlinear Discrete Systems

For a **nonlinear discrete-time system** $\mathbf{x}_{k+1} \approx \mathbf{f}_d(\mathbf{x}_k, \mathbf{u}_k)$:

We **linearize via Taylor expansion** around $(\bar{\mathbf{x}}, \bar{\mathbf{u}})$ and approximate locally as:

$$\mathbf{x}_{k+1} \approx \mathbf{f}_d(\bar{\mathbf{x}}, \bar{\mathbf{u}}) + A_d \cdot (\mathbf{x}_k - \bar{\mathbf{x}}) + B_d \cdot (\mathbf{u}_k - \bar{\mathbf{u}}), \quad A_d = \left. \frac{\partial \mathbf{f}_d}{\partial \mathbf{x}} \right|_{\bar{\mathbf{x}}, \bar{\mathbf{u}}}, B_d = \left. \frac{\partial \mathbf{f}_d}{\partial \mathbf{u}} \right|_{\bar{\mathbf{x}}, \bar{\mathbf{u}}}$$

Euler Integration (1st-order):

$$A_d = \mathcal{I} + dt \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\bar{\mathbf{x}}, \bar{\mathbf{u}}}, \quad B_d = dt \left. \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right|_{\bar{\mathbf{x}}, \bar{\mathbf{u}}}$$

Runge-Kutta 4th-order (RK4):

$$\begin{aligned} \frac{\partial \mathbf{k}_1}{\partial \mathbf{x}} &= \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\bar{\mathbf{x}}, \bar{\mathbf{u}}} & \frac{\partial \mathbf{k}_1}{\partial \mathbf{u}} &= \left. \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right|_{\bar{\mathbf{x}}, \bar{\mathbf{u}}} \\ \frac{\partial \mathbf{k}_2}{\partial \mathbf{x}} &= \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\bar{\mathbf{x}} + \frac{dt}{2} \mathbf{k}_1(\bar{\mathbf{x}}, \bar{\mathbf{u}}), \bar{\mathbf{u}}} \left(\mathcal{I} + \frac{dt}{2} \frac{\partial \mathbf{k}_1}{\partial \mathbf{x}} \right) & \frac{\partial \mathbf{k}_2}{\partial \mathbf{u}} &= \left. \frac{dt}{2} \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\bar{\mathbf{x}} + \frac{dt}{2} \mathbf{k}_1(\bar{\mathbf{x}}, \bar{\mathbf{u}}), \bar{\mathbf{u}}} \frac{\partial \mathbf{k}_1}{\partial \mathbf{u}} + \left. \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right|_{\bar{\mathbf{x}} + \frac{dt}{2} \mathbf{k}_1(\bar{\mathbf{x}}, \bar{\mathbf{u}}), \bar{\mathbf{u}}} \\ \frac{\partial \mathbf{k}_3}{\partial \mathbf{x}} &= \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\bar{\mathbf{x}} + \frac{dt}{2} \mathbf{k}_2(\bar{\mathbf{x}}, \bar{\mathbf{u}}), \bar{\mathbf{u}}} \left(\mathcal{I} + \frac{dt}{2} \frac{\partial \mathbf{k}_2}{\partial \mathbf{x}} \right) & \frac{\partial \mathbf{k}_3}{\partial \mathbf{u}} &= \left. \frac{dt}{2} \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\bar{\mathbf{x}} + \frac{dt}{2} \mathbf{k}_2(\bar{\mathbf{x}}, \bar{\mathbf{u}}), \bar{\mathbf{u}}} \frac{\partial \mathbf{k}_2}{\partial \mathbf{u}} + \left. \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right|_{\bar{\mathbf{x}} + \frac{dt}{2} \mathbf{k}_2(\bar{\mathbf{x}}, \bar{\mathbf{u}}), \bar{\mathbf{u}}} \\ \frac{\partial \mathbf{k}_4}{\partial \mathbf{x}} &= \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\bar{\mathbf{x}} + dt \mathbf{k}_3(\bar{\mathbf{x}}, \bar{\mathbf{u}}), \bar{\mathbf{u}}} \left(\mathcal{I} + dt \frac{\partial \mathbf{k}_3}{\partial \mathbf{x}} \right) & \frac{\partial \mathbf{k}_4}{\partial \mathbf{u}} &= dt \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\bar{\mathbf{x}} + dt \mathbf{k}_3(\bar{\mathbf{x}}, \bar{\mathbf{u}}), \bar{\mathbf{u}}} \frac{\partial \mathbf{k}_3}{\partial \mathbf{u}} + \left. \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right|_{\bar{\mathbf{x}} + dt \mathbf{k}_3(\bar{\mathbf{x}}, \bar{\mathbf{u}}), \bar{\mathbf{u}}} \\ A_d &= \mathcal{I} + \frac{dt}{6} \left(\frac{\partial \mathbf{k}_1}{\partial \mathbf{x}} + 2 \frac{\partial \mathbf{k}_2}{\partial \mathbf{x}} + 2 \frac{\partial \mathbf{k}_3}{\partial \mathbf{x}} + \frac{\partial \mathbf{k}_4}{\partial \mathbf{x}} \right), & B_d &= \frac{dt}{6} \left(\frac{\partial \mathbf{k}_1}{\partial \mathbf{u}} + 2 \frac{\partial \mathbf{k}_2}{\partial \mathbf{u}} + 2 \frac{\partial \mathbf{k}_3}{\partial \mathbf{u}} + \frac{\partial \mathbf{k}_4}{\partial \mathbf{u}} \right) \end{aligned}$$

Kalman Filter - What is it?

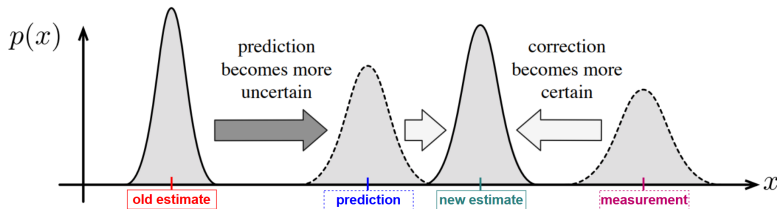
Kalman Filter (KF) was first introduced by Rudolf E. Kálmán, in his 1960 paper "*A New Approach to Linear Filtering and Prediction Problems*". KF is the **optimal bayes state estimator** for *linear system models* with **additive white Gaussian noise** in both the *motion/process* and the *observation/measurement model*. It works recursively in two stages, alternating between **Prediction** and **Correction** at each time step:

1 Prediction

The prediction step propagates the **old estimate** forward in time, using the measurement model and latest input, to arrive at the **prediction**.

2 Correction

The correction step fuses the **prediction** with the latest **measurement**, to arrive at the **new estimate**, which is done using a normalized product of the two Gaussians.



Prediction and Correction stages of Kalman Filter visualized

Kalman Filter - How it Works in Linear Systems (KF)?

Below is the Kalman Filter algorithm for **linear systems**. We assume **discrete time-dependent** state space matrices A , B , C (we removed d subscripts for simplicity). Here, we omit D as it is often zero or neglected in many physical dynamical systems (if it existed, it would only affect the *innovation* term). We assume additive white Gaussian noise $\mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, R_k)$, $\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, Q_k)$, where R_k and Q_k are the **process** and **measurement noise covariance matrices**, respectively.

① **Prediction Stage** (inputs: μ_{k-1} , $\mathbf{u}_{k-1}$, Σ_{k-1})

$$\bar{\mu}_k = A_k \mu_{k-1} + B_k \mathbf{u}_{k-1} \quad (\text{predicted state estimate})$$

$$\bar{\Sigma}_k = A_k \Sigma_{k-1} A_k^\top + R_{k-1} \quad (\text{predicted covariance estimate})$$

② **Correction Stage** (outputs: μ_k , Σ_k)

$$K_k = \bar{\Sigma}_k C_k^\top (C_k \bar{\Sigma}_k C_k^\top + Q_k)^{-1} \quad (\text{optimal Kalman gain})$$

$$\mu_k = \bar{\mu}_k + K_k \underbrace{(\mathbf{z}_k - C_k \bar{\mu}_k)}_{\text{innovation}} \quad (\text{corrected state estimate})^1$$

$$\Sigma_k = (\mathcal{I} - K_k C_k) \bar{\Sigma}_k \quad (\text{corrected covariance estimate})$$

¹ $\mu_k = (\mathcal{I} - K_k C_k) \bar{\mu}_k + K_k \mathbf{z}_k$ (linear interpolation between the predicted state estimate and the measurement)

Kalman Filter - How it Works in Nonlinear Systems (EKF)?

The **Extended Kalman Filter (EKF)** generalizes the Kalman Filter to **nonlinear systems**. We assume a **discrete-time nonlinear system** $\mathbf{x}_k = \mathbf{f}_d(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) + \mathbf{w}_{k-1}$, $\mathbf{z}_k = \mathbf{h}_d(\mathbf{x}_k) + \mathbf{v}_k$, with additive Gaussian noise $\mathbf{w}_k \sim \mathcal{N}(\mathbf{0}, R_k)$, $\mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, Q_k)$. To apply the EKF, we **linearize** the system around the current state estimates using the Jacobians: $F_k = \left. \frac{\partial \mathbf{f}_d}{\partial \mathbf{x}} \right|_{\boldsymbol{\mu}_{k-1}, \mathbf{u}_{k-1}}$, $H_k = \left. \frac{\partial \mathbf{h}_d}{\partial \mathbf{x}} \right|_{\bar{\boldsymbol{\mu}}_k}^2$.

① **Prediction Stage** (inputs: $\boldsymbol{\mu}_{k-1}$, $\mathbf{u}_{k-1}$, Σ_{k-1})

$$\bar{\boldsymbol{\mu}}_k = \mathbf{f}_d(\boldsymbol{\mu}_{k-1}, \mathbf{u}_{k-1}) \quad (\text{predicted state estimate})$$

$$\bar{\Sigma}_k = F_k \Sigma_{k-1} F_k^\top + R_{k-1} \quad (\text{predicted covariance estimate})$$

② **Correction Stage** (outputs: $\boldsymbol{\mu}_k$, Σ_k)

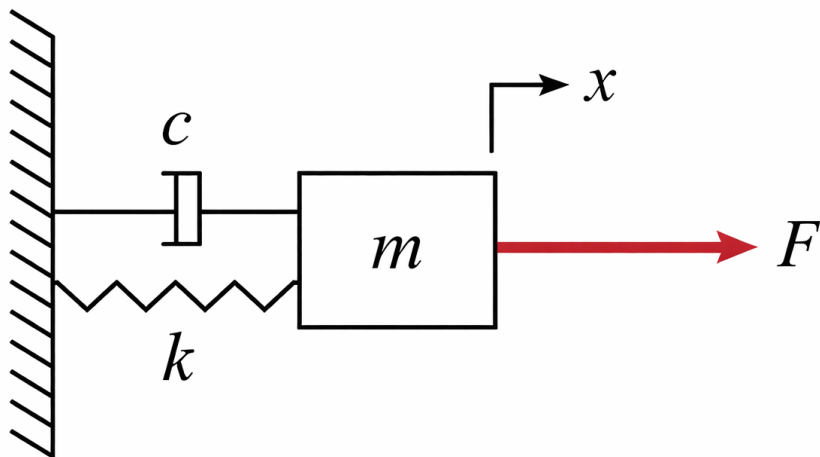
$$K_k = \bar{\Sigma}_k H_k^\top (H_k \bar{\Sigma}_k H_k^\top + Q_k)^{-1} \quad (\text{near-optimal Kalman gain})$$

$$\boldsymbol{\mu}_k = \bar{\boldsymbol{\mu}}_k + K_k \underbrace{(\mathbf{z}_k - \mathbf{h}_d(\bar{\boldsymbol{\mu}}_k))}_{\text{innovation}} \quad (\text{corrected state estimate})$$

$$\Sigma_k = (\mathcal{I} - K_k H_k) \bar{\Sigma}_k \quad (\text{corrected covariance estimate})$$

² they substitute the matrices A_k and C_k respectively of the linear case KF

Damped Harmonic Oscillator Exercise



Damped Harmonic Oscillator System

Damped Harmonic Oscillator Exercise - System Dynamics

The **Damped Harmonic Oscillator** is a second order **linear system**, that consists of a *double integrator* tied to a *spring*, which provides a restoring force toward a stable equilibrium (point or limit cycle), and a *damper*, which dissipates energy and reduces the motion over time. We assume:

- parameters: m (mass), c (damping coefficient), k (spring constant)
- state: $\mathbf{x} = \begin{bmatrix} x \\ \dot{x} \end{bmatrix} \in \mathbb{R}^2$ contains the position and velocity of mass m
- control: $u = F \in \mathbb{R}$ is the scalar driving force of the system

We measure only the position, so the continuous-time state space model is:

$$\ddot{x} + \frac{c}{m}\dot{x} + \frac{k}{m}x = \frac{u}{m} \quad \Rightarrow \quad \begin{array}{l} \dot{\mathbf{x}} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix} \mathbf{x} + \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix} u \\ y = \begin{bmatrix} 1 & 0 \end{bmatrix} \mathbf{x} \end{array}$$

Since this is a **linear system**, it can be **solved analytically**.

Damped Harmonic Oscillator Exercise - Discretized System

Since we have a linear system, its **discretization can be done analytically**, using for example **zero-order hold**. The *continuous-time state space model* of the system is:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad \mathbf{D} = 0$$

So, the *discrete-time state space model* is analytically derived to be:

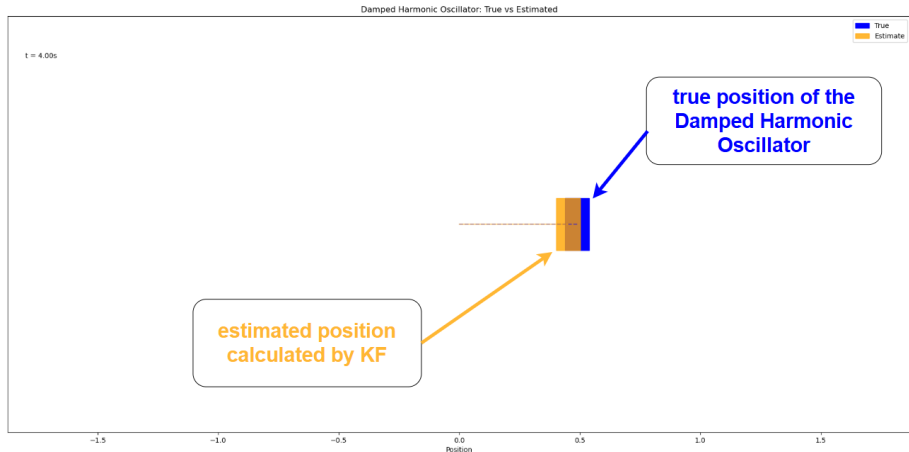
$$\mathbf{A}_d = \frac{m}{\sqrt{c^2 - 4km}} \begin{bmatrix} \lambda_2 e^{\lambda_1 dt} - \lambda_1 e^{\lambda_2 dt} & e^{\lambda_2 dt} - e^{\lambda_1 dt} \\ \lambda_1 \lambda_2 (e^{\lambda_1 dt} - e^{\lambda_2 dt}) & \lambda_2 e^{\lambda_2 dt} - \lambda_1 e^{\lambda_1 dt} \end{bmatrix}, \quad \lambda_{1,2} = \frac{-c \mp \sqrt{c^2 - 4km}}{2m} \in \mathbb{C}$$

$$\mathbf{B}_d = \mathbf{A}^{-1}(\mathbf{A}_d - \mathbf{I})\mathbf{B}, \quad \mathbf{A}^{-1} = \frac{m}{k} \cdot \begin{bmatrix} -\frac{c}{m} & -1 \\ \frac{k}{m} & 0 \end{bmatrix}, \quad \mathbf{C}_d = \mathbf{C} = \begin{bmatrix} 1 & 0 \end{bmatrix}$$

We could also use the more easily derived *linear approximations*:

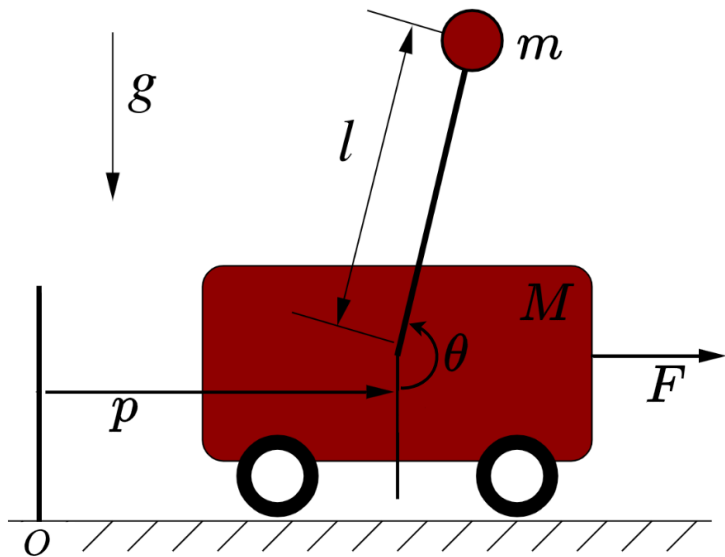
$$\mathbf{A}_d = e^{\mathbf{A}dt} \approx \mathbf{I} + \mathbf{A}dt, \quad \mathbf{B}_d = \mathbf{A}^{-1}(e^{\mathbf{A}dt} - \mathbf{I})\mathbf{B} \approx \mathbf{B}dt$$

Damped Harmonic Oscillator Exercise - Example



Damped Harmonic Oscillator animation: true position vs KF estimated position

Cart-Pole Exercise



Cart-Pole System

Cart-Pole Exercise - System Dynamics

The **Cart-Pole** system consists of a *translating cart* (a *double integrator* under applied force) coupled to an *inverted pendulum*. This system introduces **nonlinear dynamics** and an unstable equilibrium. We assume:

- parameters: m (pendulum mass), M (cart mass), l (pole length)
- state: $\mathbf{x} = \begin{bmatrix} p & \dot{p} & \theta & \dot{\theta} \end{bmatrix}^T \in \mathbb{R}^4$ contains the position and velocity of the cart, and the angular position and velocity of the pole
- control: $u = F \in \mathbb{R}$ is the scalar driving force of the system

We can derive its continuous-time state space model using Lagrangian mechanics (we assume that only the two position variables are observed):

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{p} \\ \ddot{p} \\ \dot{\theta} \\ \ddot{\theta} \end{bmatrix} = \begin{bmatrix} \dot{p} \\ \frac{m \sin \theta (l \dot{\theta}^2 + g \cos \theta) + u}{M + m \sin^2 \theta} \\ \dot{\theta} \\ \frac{-m l \dot{\theta}^2 \sin \theta \cos \theta - (m + M) g \sin \theta - u \cos \theta}{l (M + m \sin^2 \theta)} \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \mathbf{x}$$

Cart-Pole Exercise - Discretized System & Jacobians

We already have showed that in order to derive the *discrete-time state space* matrices $F = A_d$ and B_d , ($H = C_d = C$ is constant), we need to first compute the Jacobians $A = \frac{\partial \mathbf{f}}{\partial \mathbf{x}}$ and $B = \frac{\partial \mathbf{f}}{\partial \mathbf{u}}$ of the *continuous-time system*. Here, we get:

$$A = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial \mathbf{f}}{\partial p} & \frac{\partial \mathbf{f}}{\partial \dot{p}} & \frac{\partial \mathbf{f}}{\partial \theta} & \frac{\partial \mathbf{f}}{\partial \dot{\theta}} \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{\partial f_2}{\partial \theta} & \frac{\partial f_2}{\partial \dot{\theta}} \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{\partial f_4}{\partial \theta} & \frac{\partial f_4}{\partial \dot{\theta}} \end{bmatrix}, \quad B = \frac{\partial \mathbf{f}}{\partial u} = \begin{bmatrix} 0 \\ \frac{1}{M+m \sin^2 \theta} \\ 0 \\ \frac{-\cos \theta}{I(M+m \sin^2 \theta)} \end{bmatrix}$$

Where:

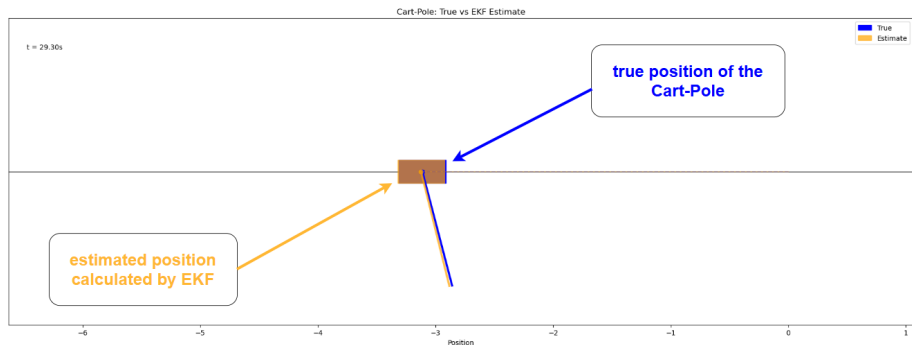
$$\frac{\partial f_2}{\partial \theta} = \frac{m(M - m \sin^2 \theta)(g + l\dot{\theta}^2 \cos \theta) - 2mMg \sin^2 \theta - m \sin(2\theta)u}{(M + m \sin^2 \theta)^2}$$

$$\frac{\partial f_2}{\partial \dot{\theta}} = \frac{2ml\dot{\theta} \sin \theta}{M + m \sin^2 \theta}$$

$$\frac{\partial f_4}{\partial \theta} = \frac{-(ml\dot{\theta}^2 + g(m + M) \cos \theta)(M - m \sin^2 \theta) + 2mMl\dot{\theta}^2 \sin^2 \theta + \sin \theta(m + m \cos^2 \theta + M)u}{I(M + m \sin^2 \theta)^2}$$

$$\frac{\partial f_4}{\partial \dot{\theta}} = -\frac{m\dot{\theta} \sin(2\theta)}{M + m \sin^2 \theta}$$

Cart-Pole Exercise - Example



Cart-Pole animation: true position vs EKF estimated position

- **State Estimation (wikipedia's article, control point of view)**
- **Discretization (wikipedia's article)**
- **Kalman Filter (wikipedia's article)**
- **Extended Kalman Filter (wikipedia's article)**
- **The Physics of the Damped Harmonic Oscillator**
- **The Physics of Cart-Pole**
- **State Estimation for Robotics; Timothy Barfoot (chapters 2, 3)**
- **Probabilistic Robotics; Sebastian Thrun, Wolfram Burgard, Dieter Fox (sections 2.1-2.5, 3.1, 3.2, 5.1-5.4, 6.3)**
- **Modern Robotics: Mechanics, Planning, and Control; Kevin M. Lynch and Frank C. Park**